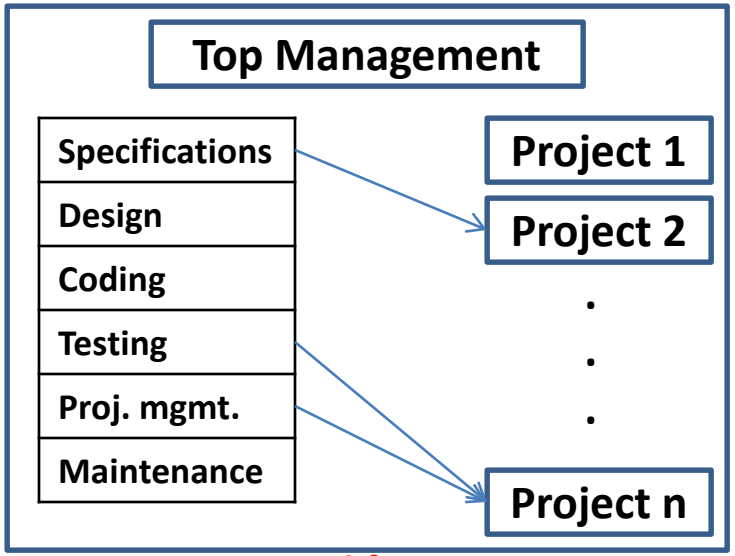
Organization Structures

Employees in S/W organization are often categorized in to different verticals or groups. The grouping differs among companies depending on type of service they provide.

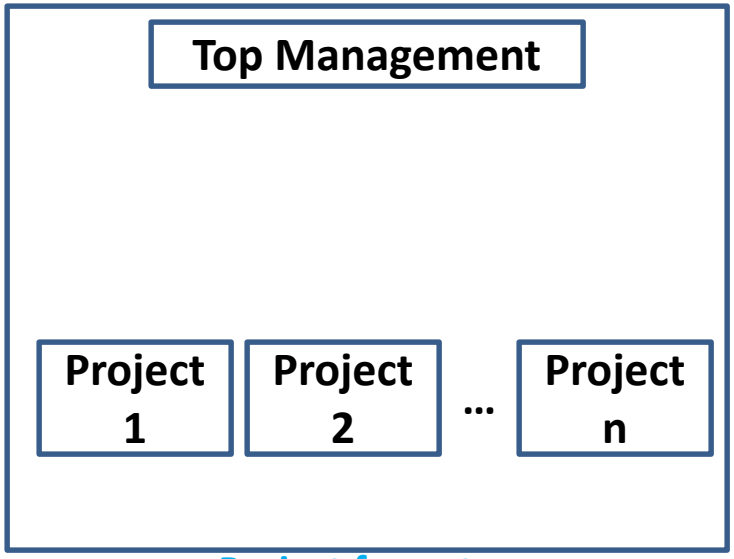
S. S. Satapathy

Organization Structures

- **Functional format**
  - the development staff are divided based on the functional group to which they belong. The different projects borrow engineers from the required functional groups for specific phases to be undertaken in the project and return them to the functional group upon the completion of the phase.
  - **Advantages: Ease of staffing; Production of good quality documents; Job specialization; Efficient handling of the problems associated with manpower turnover**
- **Project format**
  - In the project format, the project development staff are divided based on the project for which they work.
  - **Preferred in smaller organization because of ease in job rotation** among the project team member.



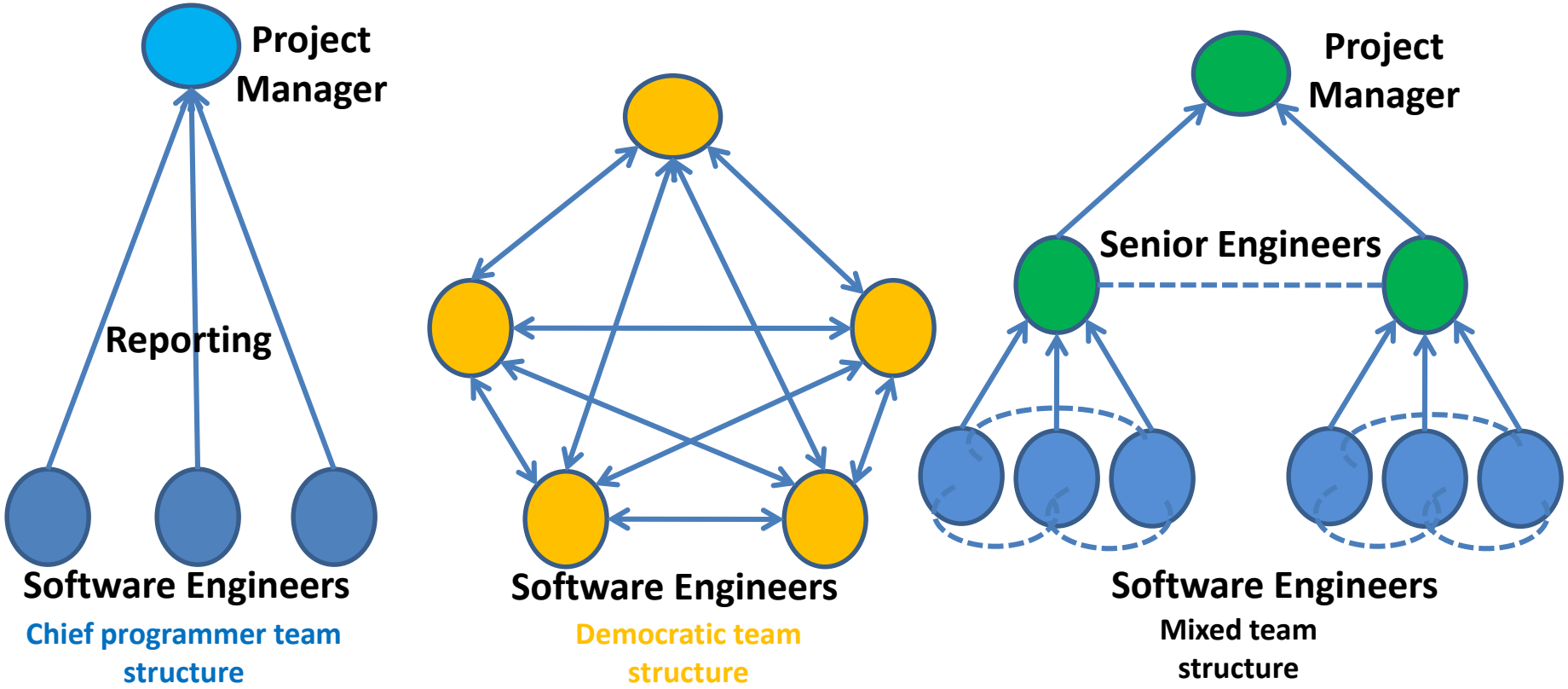
Functional format



Project format

Team Structures

- Chief programmer
  - A hierarchy is maintained among the members
- Democratic
  - No formal team hierarchy. At different times, different members of the group provide technical leadership.
- Mixed team organizations



Characteristics of a good software engineer

- **Exposure to systematic techniques**: familiarity with software engineering principles.
- **Domain knowledge**: Good technical knowledge of the project areas.
- **Good programming abilities**.
- **Good communication skills**. These skills comprise of oral, written, and interpersonal skills.
- **High motivation**.
- **Sound knowledge of fundamentals of computer science**.
- **Intelligence**.
- **Ability to work in a team**.
- **Discipline**, etc.

Risk Management

Software development is often associated with different risks. Proper management of risk is essential to develop software in time with profit.

S. S. Satapathy

Risk Management

- A software project can be affected by a large **variety of risks**. There are **three main categories of risks** those can affect a software project:
- **Project risks:**
  - Project risks concern varies forms of **budgetary**, **schedule**, **personnel**, **resource**, and **customer-related problems**.
  - An important project risk is **schedule slippage (failure to meet a standard or deadline)**. Unlike any manufacturing project, the project manager can not physically visualize the software product. Thus it is not easy to assess the progress of the work and control it.
- **Technical risks:**
  - Technical risks concern potential **design**, **implementation**, **interfacing**, **testing**, and **maintenance** problems.
  - Technical risks **also include ambiguous specification**, **incomplete specification**, **changing specification**, **technical uncertainty**, and **technical obsolescence**.
  - Most technical risks occur due to the **development team's insufficient knowledge** about the project.
- **Business risks:**
  - This type of risks include risks of **building an excellent product that no one wants to purchase**, losing budgetary or personnel commitments, etc.

Risk Assessment

- The **objective** **of risk assessment** is to **rank the risks in terms of their damage causing potential**. For risk assessment, first each risk should be rated in two ways:
  - The **likelihood of a risk becoming true** (**denoted as r**).
  - The **consequence of the problems associated with that risk** OR the **sensitivity of that risk** (**denoted as s**).
  - Based on these two factors, the **priority** **of each risk (p)** can be computed as:

$$p = r * s$$

  - If all identified risks are prioritized, then the **most likely and damaging risks can be handled first** and more comprehensive risk abatement procedures can be designed for these risks.

Risk containment

- After all the identified risks of a project are assessed, **plans must be made to contain** the **most damaging** and the **most likely risks**.
- **Different risks require different containment procedures**. In fact, most risks require ingenuity on the part of the **project manager in tackling the risk**.
- There are **three main strategies** to plan for **risk containment**:
  - **Avoid the risk:** This may take several forms such as **(i)** **discussing with the customer** **to change the requirements** to reduce the scope of the work, **(ii)** **giving incentives to the engineers** **to avoid the risk of manpower turnover**, etc.
  - **Transfer the risk:** This strategy involves **(i)** **getting the risky component developed by a third party**, **(ii)** **buying insurance cover**, etc.
  - **Risk reduction:** This involves planning ways to contain the damage due to a risk. For example, if there is **risk that some key personnel might leave**, **new recruitment may be planned**.

Risk Leverage

- To choose between the **different strategies of handling a risk**, the project manager must consider the **cost of handling the risk** and the **corresponding reduction of risk**. For this the **risk leverage** of the different risks can be computed.
- **Risk leverage** is the **difference** in **risk exposure divided by** the **cost of reducing the risk**. More formally,
- **risk leverage = (risk exposure before reduction – risk exposure after reduction) / (cost of reduction)**

Risk related to schedule slippage

- Risks relating to schedule slippage arise **(failure to meet a standard or deadline)** primarily **due to the intangible nature of software product**.
- Therefore, these can be **dealt with by increasing the visibility of the software product**.
  - Visibility of a software product can be increased by
  - **Producing relevant documents** **during the development process** wherever meaningful and getting these documents reviewed by an appropriate team.
  - **Milestones should be placed** at regular intervals through a software engineering process to provide a manager with regular indication of progress.
  - **Divide every phase** **in to reasonable-sized tasks and schedule milestones for these tasks too**.
    - A milestone is reached, once documentation produced as part of a software engineering task is produced and gets successfully reviewed. Milestones need not be placed for every activity. An approximate rule of thumb is to **set a milestone every 10 to 15 days**.